¡¡COLLEGE¿¿

Final Year Project Report

GIST: Detecting, Classifying and Localizing GNSS Jamming and Spoofing from Crowdsourced ADS-B with Deep Learning

Submitted by
¡¡AUTHOR¿¿ (¡¡ROLL_NO¿¿)

Under the guidance of
¡¡GUIDE¿¿

¡¡DEPARTMENT¿¿
¡¡MONTH YEAR¿¿

Certificate

[TODO: College certificate text, signed by the guide and head of department.]

Acknowledgements

[TODO: Acknowledgements.] The ADS-B data used in this work is provided by ADSB.lol and its contributors under the Open Database License.

Abstract

GNSS (GPS) interference affects thousands of flights every day in several regions of the world. Two different threats are involved: *jamming*, which denies the satellite signal, and *spoofing*, which replaces it with a false one. Public monitoring maps threshold the Navigation Integrity Category (NIC) broadcast by aircraft; this detects jamming but misses spoofing, since a spoofed receiver reports high integrity. This project builds GIST, a deep learning system that treats every aircraft as a remote GNSS sensor. It derives physics-consistency features that a spoofer cannot fake cheaply, builds weak labels from labeling functions and a generative label model so that no manual annotation is needed, trains a trajectory Transformer with spatial graph attention to classify each aircraft-window as normal, jammed or spoofed, and localizes jammers with a differentiable radio-propagation layer trained on simulated emitters injected into real traffic. GIST reaches a spoofing recall of – at 1% false-positive rate on real 2026 traffic, where an integrity-threshold monitor reaches 0.01, and localizes simulated jammers among benign low-integrity aircraft with a median error of – km versus 42.5 km for non-linear least squares.

Contents

1 Introduction **8**
1.1 Motivation 8
1.2 Problem statement 8
1.3 Objectives 8
1.4 Organisation of the report 9

2 Background **10**
2.1 GNSS and its vulnerabilities 10
2.2 ADS-B 10
2.3 Why integrity thresholds miss spoofing 10
2.4 Deep learning components 11

3 Literature Review **12**

4 Data and Weak Supervision **13**
4.1 Data source and licence 13
4.2 Processing 13
4.3 Features 13
4.4 Labeling functions and the label model 13

5 The GIST Model **15**

6 Interference Simulator **16**

7 Implementation **17**
7.1 Software stack 17
7.2 Hardware constraints 17

8 Results **18**
8.1 Case studies 18

9 Challenges and Lessons **20**

10 Conclusion and Future Work **21**

A Reproducing the results **22**

List of Figures

List of Tables

4.1 Labeling functions (window = 10 min). "modern" = ADS-B version ≥ 1; "cruise" = median altitude $> 10\,000$ ft. 14

8.1 Test split (2026 days). Detection metrics per aircraft-window: macro-F1 over 3 classes, average precision (AP) and spoofing recall at 1% false-positive rate (R@1%). Real = label-model targets; Sim = exact simulated truth. Single-score baselines cannot separate jamming from spoofing; their anomaly score is used for both. Mean±std over seeds. 19

8.2 Ablations on the test split (seed 0 unless noted). 19

Chapter 1

Introduction

1.1 Motivation

Since 2022, interference with GNSS signals has become routine around the Baltic Sea, the Eastern Mediterranean, the Black Sea, Iraq and Iran, Myanmar and the India–Pakistan border. Aircraft lose their satellite position, or worse, are fed a confident but false one. Air navigation service providers, airlines and regulators need to know where interference happens, whether it is jamming or spoofing, and where it comes from.

1.2 Problem statement

Given crowdsourced ADS-B broadcasts from all aircraft in a region, (1) detect GNSS interference at the level of individual aircraft and 10-minute windows, (2) classify it as jamming or spoofing, and (3) estimate the location of jammers, without manually labelled training data.

1.3 Objectives

1. Build an open, reproducible data pipeline from public ADS-B archives.
2. Design features that remain informative when the GNSS solution itself is manipulated.
3. Produce training labels by weak supervision.
4. Train a deep model that uses both each aircraft's history and its neighbours.
5. Localize jammers and compare with published classical methods.
6. Provide an interactive map for inspection.

1.4 Organisation of the report

Chapter 2 gives background on GNSS and ADS-B. Chapter 3 reviews related work. Chapter 4 describes the data and weak supervision, Chapter 5 the model, Chapter 6 the simulator, Chapter 7 the implementation, Chapter 8 the results and Chapter 9 the challenges met. Chapter 10 concludes.

Chapter 2

Background

2.1 GNSS and its vulnerabilities

GNSS receivers compute position from signals that arrive at the Earth's surface at around $-160\,\mathrm{dBW}$, far below the thermal noise floor. A transmitter of a few watts can therefore overwhelm the signal over hundreds of kilometres (jamming). A spoofer broadcasts counterfeit signals that the receiver locks onto, producing a wrong position or time.

2.2 ADS-B

Automatic Dependent Surveillance–Broadcast (ADS-B) is the system by which aircraft broadcast their GNSS-derived position, velocity and identity about twice per second on 1090 MHz. Messages also carry integrity indicators: the Navigation Integrity Category (NIC), Navigation Accuracy Category for position ($\mathrm{NAC_p}$), Source Integrity Level (SIL) and Geometric Vertical Accuracy (GVA). Aircraft also report barometric altitude, which does not depend on GNSS, and geometric (GNSS) altitude. Volunteer receiver networks such as adsb.lol and the OpenSky Network collect these broadcasts worldwide. Multilateration (MLAT) between ground receivers gives an independent position for aircraft seen by several receivers.

2.3 Why integrity thresholds miss spoofing

Under jamming the receiver loses track and NIC falls towards zero. Under spoofing the receiver tracks a consistent but false signal and continues to report a high NIC. Detecting spoofing therefore requires checking the reported position against information the spoofer does not control.

2.4 Deep learning components

We use the Transformer encoder [**?**] for per-aircraft sequences and attention over nearby aircraft in the style of graph attention networks [**?**]. Weak supervision follows the data-programming approach [**?**] with a Dawid–Skene label model [**?**].

Chapter 3

Literature Review

Liu et al. [**?**, **?**] locate jammers with ADS-B by modelling received power with the Friis equation and solving a non-linear least squares problem; on the Denver 2022 event they report about 4 km error. Schäfer et al. [**?**] use radio horizons to bound the jammer position. gpsjam.org [**?**] publishes daily maps of the share of aircraft with low NIC per hexagon. The DLR study [**?**] measures global interference from space-based ADS-B, and an industry white paper [**?**] describes spoofing checks based on integrity and position consistency. None of these combine learned jam/spoof classification, cross-aircraft evidence and learned localization with weak supervision, which is the gap this project addresses.

Chapter 4

Data and Weak Supervision

4.1 Data source and licence

The adsb.lol `globe_history` archive [**?**] publishes one tarball of readsb trace files per day under ODbL-1.0. ADS-B Exchange sample data was considered and rejected because its terms limit it to evaluation. We use 5 days (1st of each month, January 2024 to October 2026), 67 M airborne trace points inside the study regions and 1.22 M aircraft-windows.

4.2 Processing

Trace points are parsed in parallel and filtered to six interference regions and three comparison regions. Integrity fields, which appear only in periodic detail objects, are forward-filled with their age recorded. Points are binned into 30-second steps and 10-minute windows.

4.3 Features

[TODO: Copy and extend Section III-B of the paper; include the wind-estimation fix from Chapter 9.]

4.4 Labeling functions and the label model

[TODO: Describe the label model, the all-abstain issue and its fix, and per-region label statistics.]

Table 4.1: Labeling functions (window = 10 min). "modern" = ADS-B version ≥ 1; "cruise" = median altitude $> 10\,000$ ft.

LF	Vote	Condition
nic_zero	JAM	modern, cruise, $\geq 30\%$ steps NIC ≤ 1
nic_drop	JAM	modern, cruise, NIC ≥ 8 seen and $\geq 30\%$ steps NIC ≤ 6
pos_loss	JAM	cruise, ADS-B gap ≥ 90 s, $\geq 10\%$ steps NIC ≤ 6
cell_jam	JAM	$\geq 30\%$ of cell aircraft and $\geq 30\%$ own steps NIC ≤ 6
teleport	SPOOF	position jump > 10 km with NIC ≥ 7
gdiff	SPOOF	≥ 2 steps with $\lvert\Delta h_{geo-baro} - \text{nbr}\rvert > 1000$ ft, NIC ≥ 7
cluster	SPOOF	member of a co-located, altitude-spread cluster
mlat[†]	SPOOF	MLAT vs. ADS-B position disagreement > 10 km
clean	NORMAL	all NIC ≥ 8, no jump/gdiff/cluster, ≥ 10 steps
clean_lowcell	NORMAL	modern, cell NIC-low share $< 5\%$, own NIC never low

[†]held out of training labels; evaluation only.

Chapter 5

The GIST Model

[TODO: Extend Section IV of the paper with an architecture diagram and the loss functions.]

Chapter 6

Interference Simulator

[TODO: Extend Section V of the paper with the propagation model, antenna pattern and the EIRP calibration.]

Chapter 7

Implementation

7.1 Software stack

Python 3.12, PyTorch with the Apple MPS backend, Polars and DuckDB for data processing, H3 for spatial indexing, NumPy and SciPy for the baselines. All code is in a single repository with tests for propagation and the label model.

7.2 Hardware constraints

All experiments ran on an Apple M3 laptop with 8 GB of memory. This drove several design choices: streaming parse with raw data deleted after processing, a 21×21 localization grid with coarse-to-fine refinement instead of a dense grid, padding scenes to four fixed sizes to avoid repeated GPU graph compilation, and capping the GPU memory cache.

Chapter 8

Results

8.1 Case studies

[TODO: Insert case-study figures from paper/figures (case_*.pdf) with discussion.]

Table 8.1: Test split (2026 days). Detection metrics per aircraft-window: macro-F1 over 3 classes, average precision (AP) and spoofing recall at 1% false-positive rate (R@1%). Real = label-model targets; Sim = exact simulated truth. Single-score baselines cannot separate jamming from spoofing; their anomaly score is used for both. Mean±std over seeds.

	Real traffic				Simulated emitters			
Method	F1	Jam AP	Spoof AP	Spoof R@1%	F1	Jam AP	Spoof AP	Spoof R@1
gpsjam-style threshold	0.62	0.86	0.01	0.01	0.26	0.93	0.07	0.01
Isolation Forest	0.56	0.80	0.04	0.06	0.27	0.48	0.15	0.01
LSTM autoencoder	0.38	0.37	0.03	0.06	0.27	0.58	0.13	0.03
GIST (0 seeds)	–	–	–	–	–	–	–	–

	Clean scenes		+5% benign low-NIC aircraft	
Jammer localization [km]	median	p95	median	p95
Centroid of affected	105.0	260.2	126.4	266.4
Radio-horizon score	121.3	382.3	138.0	448.6
NLS (Liu et al.)	26.0	301.7	42.5	315.4
GIST	–	–	–	–

Table 8.2: Ablations on the test split (seed 0 unless noted).

	Real		Sim		Loc.
Variant	F1	Spoof R@1%	F1	Spoof R@1%	med. km
GIST (full)	–	–	–	–	–
w/o spatial graph attention	–	–	–	–	–
integrity fields only	–	–	–	–	–
w/o simulated training scenes	–	–	–	–	–

Chapter 9

Challenges and Lessons

- **Data licensing.** The first-choice dataset (ADS-B Exchange samples) was evaluation-only; switching to adsb.lol (ODbL) kept the work publishable and gave daily rather than monthly coverage.
- **Download throttling.** GitHub throttled release downloads to a few bytes per second after about 7 GB; the downloader was rewritten to resume per part and back off.
- **Noisy wind feature.** A naive wind triangle from forward-filled airspeed and heading was useless (99th percentile of several hundred knots in clean regions). Using readsb's own wind estimate only on points with fresh data, excluding turns and comparing within altitude bands reduced the median deviation to about 5 kt.
- **Label-model artifact.** Windows on which every labeling function abstained were assigned to the class with the highest abstention rate (spoofing), producing 13% "spoofed" windows over Australia. These windows are now unlabelled.
- **Unrealistic first simulator.** Emitters of 1–100 W affected 0.4% of aircraft per scene; raising the power to the range estimated in the literature gave realistic affected regions.
- **Memory on an 8 GB laptop.** The training process grew to 7.6 GB through the GPU caching allocator, slowing epochs by an order of magnitude until the cache was capped.

Chapter 10

Conclusion and Future Work

[TODO: Summarise results.] Future work: the Denver 2022 case study with OpenSky data (pending data access), the MLAT-only release as an independent position reference, receiver-aware radio-horizon checks, and an online version that updates every few minutes.

Appendix A

Reproducing the results

See `README.md`: fetch days, build labels, build scene datasets, train, evaluate, generate tables and figures.